Draft version July 13, 2026
Typeset using LaTeX **twocolumn** style in AASTeX7.0.1

The Barnes-Hut Gravity Tree on an FPGA

Angus Beane [1, 2] and others[3]

[1] *Center for Cosmology and Particle Physics, Department of Physics, New York University, 726 Broadway, New York, NY 10003, USA*
[2] *Simons Society of Fellows, Simons Foundation, New York, NY 10010, USA*
[3] *Places*

ABSTRACT

In the N-body problem, solving for the gravitational force at a given position via direct summation is intractable for large particle numbers due to its $\mathcal{O}\left(N^2\right)$ complexity. The Barnes-Hut (BH) tree is one of several approximate methods with $\mathcal{O}\left(N \log N\right)$ complexity or better, enabling simulations of galaxies, cosmological boxes, star clusters, and many others where self-gravity is important. However, the irregular memory access of such approaches makes them less conducive to hardware acceleration on devices such as field programmable gate arrays (FPGAs) and application-specific integrated circuits (ASICs). We introduce the first BH tree algorithm in which both tree construction and walking occur on a custom hardware device. After sorting particles along a space-filling curve, the tree is constructed in a single streaming pass and stored as a flat array. Then, for a given particle, tree walking is a single forward pass through this array, with skips. We implement this algorithm on an FPGA as pure-HLS kernels, with a cyclic dataflow region for tree walking. Both tree construction and walking can be pipelined with an initiation interval of 1. While the relative lack of progress in FPGA development over the last decade means our current algorithm is not competitive with current GPUs, it does lay the groundwork for tree-specific ASIC development along the lines of GRAPE boards, which could be competitive in energy efficiency and potentially performance. Our source code is publicly available.

Keywords: Galaxies (573) — Cosmology (343) — High Energy astrophysics (739) — Interstellar medium (847) — Stellar astronomy (1583) — Solar physics (1476)

1. INTRODUCTION

Many astrophysical systems like stars, galaxies, and the universe have significant self-gravity, and so modeling their evolution in numerical simulations requires solving for the gravitational force at an arbitrary position in space. Such systems are discretized, with a continuous distribution of mass divided amongst a finite number of N bodies typically treated as softened point masses at positions $\mathbf{x}_j$. The direct computation of the force at a position $\mathbf{x}$ is then given by,

$$\mathbf{a}\left(\mathbf{x}\right) = \sum_{j=1}^{N} G m_j W(|\mathbf{x_j} - \mathbf{x}|)\hat{r}, \tag{1}$$

where $\hat{r} = \left(\mathbf{x_j} - \mathbf{x}\right) / |\mathbf{x_j} - \mathbf{x}|$, and $W(r)$ is the softening kernel. In the Newtonian, non-softened case $W(r) = 1/r^2$.

To compute the self-gravity of such a system (i.e., to compute the force at the position of each of the N particles) requires the computation of $N\left(N-1\right)/2$ non-trivial interactions, each of which involves an expensive `sqrt` call. This $\mathcal{O}\left(N^2\right)$ makes the direct computation of the gravitational force prohibitively expensive for systems involving more than $\sim 10^6$ particles.

As a result, many approximate methods for gravity calculations have been developed with much more attractive scalings. The first is the Barnes-Hut (BH) tree algorithm (J. Barnes & P. Hut 1986). In this method, the particle distribution is first divided into a hierarchical oct-tree (a quad-tree in 2D), which is subdivided until each node of the tree has holds a number of particles fewer than some small threshold, or up to some maximum depth. The multipole moments of each oct-tree node are computed. To compute the acceleration at a position, one assesses the accuracy of the multipole approximation for each top-level node. If this accuracy is deemed insufficient, that node is "opened," meaning all of its subnodes are assessed in the same way. This process is repeated until all nodes have been exhausted.

For a single force calculation, the number of interactions at each level of the tree remains roughly constant,

Corresponding author: Angus Beane
Email: angus.beane@nyu.edu

and so the typical cost scales with the depth of the tree, which scales as $\log N$. Thus, for a system of particles, the total force calculation scales as $\mathcal{O}(N \log N)$. In astrophysics, the BH tree is widely adopted, underlying a substantial fraction of recent work in galaxy formation and cosmology (e.g., P. F. Hopkins et al. 2018; A. Pillepich et al. 2018; R. Davé et al. 2019).

An improved technique is the fast multipole method (FMM; L. Greengard & V. Rokhlin 1987; H. Cheng et al. 1999). Here, instead of evaluating particle-node interactions, node-node pairs expand each other's multipole moments locally. These local expansions are accumulated for all distant nodes. The force on each particle is evaluated only once, after all distant nodes have been accounted for. This leads to an attractive scaling of $\mathcal{O}(N)$, and it has been applied in astrophysics (W. Dehnen 2000, 2014; V. Springel et al. 2021; M. Schaller et al. 2024), albeit at the cost of a higher algorithmic complexity.

A tree or FMM solver is often augmented with particle-mesh (PM) gravity solvers (R. W. Hockney & J. W. Eastwood 1981; A. A. Klypin & S. F. Shandarin 1983; S. D. M. White et al. 1983; V. Springel 2005; M. Vogelsberger et al. 2020). In the PM method, the discrete mass distribution is placed onto a grid through some kernel function, Fourier transformed, multiplied by the relevant Green's function, and transformed back to real space to recover an approximate field potential. This potential can then be differentiated through standard methods in order to recover the force at a given point. While this technique can readily make use of the Fast Fourier Transform, a prohibitively fine grid is needed for the highly clustered systems common in astrophysics. For this reason, a short-range and long-range split is used, where contribution from long-range sources is handled by PM and from short-range forces by a tree, FMM, or direct summation method (G. Xu 1995; V. Springel 2005; L. H. Garrison et al. 2021). TODO: lehman citation is wrong

Significant effort has been made to accelerate force calculations on general purpose hardware. Direct summation is an excellent match to the single instruction multiple threads (SIMT) architecture on GPUs. Each thread performs the same (or very similar) operations for each interaction, and the data is accessed in large, contiguous chunks, necessary to maximize bandwidth. For this reason, direct summation has been readily implemented on CPUs (S. von Hoerner 1960; S. J. Aarseth 1999, 2003), GPUs (L. Nyland et al. 2004; S. F. Portegies Zwart et al. 2007; R. G. Belleman et al. 2008; L. Wang et al. 2015), ASICs (D. Sugimoto et al. 1990; S. K. Okumura et al. 1993; J. Makino et al. 2003), and FPGAs (T. Hamada et al. 2000; K. Sano et al. 2017).[4]

However, the BH tree does not map as cleanly to hardware accelerators because of the irregular memory access patterns of tree construction and walking. Early attempts overcame this by constructing and walking the tree on a host CPU to build an interaction list and then performing the actual interactions on a GPU (T. Hamada et al. 2009; E. Gaburov et al. 2010), ASIC (J. Makino 1991), or FPGA (A. Kawai & T. Fukushige 2006).

On GPUs, J. Bédorf et al. (2012) later demonstrated a modified tree algorithm called `Bonsai` that can run entirely on the GPU. By constructing the tree level-by-level, and walking the tree breadth-first for groups of particles, `Bonsai` is able to reduce most of construction/walking to GPU-friendly algorithms like radix sort and stream compaction. `Bonsai` was successfully run on 18,600 GPUs across the same number of nodes with a reported efficiency of 86% relative to a single GPU (J. Bédorf et al. 2014). As they discuss, running entirely on the GPU is absolutely critical when particles have individual timesteps, since timesteps where few particles are active can be dominated by communication between the host and device.

In this work, we introduce the first, to our knowledge, implementation of the BH tree on an FPGA in which both tree construction and walking are performed entirely on the board. In Section 2, we summarize the existing BH tree implementations. In Section 3, we detail our new tree walking algorithm and explain why it is better suited to custom hardware than existing implementations. Then, in Section 4 we discuss specific details of how the algorithm was implemented on our FPGA. In Section 5 we summarize the performance characteristics of our design. We then compare our performance to CPUs and GPUs, as well as future improvements, in Section 6. Finally, we conclude in Section 7.

2. CURRENT IMPLEMENTATIONS

We now first describe the algorithm in general and then summarize two of the more common current implementations. Implementations of the algorithm mainly differ in how the tree data is constructed and stored in memory.

2.1. *Barnes-Hut Algorithm*

The main idea behind the BH algorithm is to account for the partial force of distant particles by aggregating their effect together (J. Barnes & P. Hut 1986). In par-

[4] And lightbulbs (E. Holmberg 1941).

ticular, one divides the domain into an octree (quadtree in 2D) and computes the multipole moments (monopole, quadrupole, etc.) of all particles in each spatial subdivision at each level. These subdivisions are referred to as nodes.[5] This can be done down to a fixed maximum depth, or the spatial domain can be arbitrarily divided until each node contains fewer than a threshold number of particles (typically $\sim$ 1–10). Nodes that are not subdivided further (i.e., they have no children) are referred to as leaves.

Once the tree is built, forces are computed at a given point by recursively "walking" the tree. Starting at the top-level node, one either accepts the multipole approximation and adds its partial force contribution, or rejects it and applies the same consideration to all of its children nodes. This, of course, necessitates a so-called multipole acceptance criterion (MAC). The original MAC was a geometric one, in which the opening angle $\theta = l/d$ is computed, where l is the size of the node and d is the distance to the node's center. If this angle is less than a threshold value (typically $\sim$ 0.1–0.7), then the node's multipole approximation is accepted.[6] Accepting the MAC for a node is referred to as closing the node, and rejecting it is referred to as opening the node.

Note that the algorithm is inherently irregular during tree walking, as it cannot be determined in advance which nodes will be opened or closed, and the precise path will vary based on the target position. Furthermore, depending on the layout of the tree, node closures can result in quite large jumps in memory.

2.2. *Pointer Chasing*

Early implementations of the BH tree, and many subsequent codes, used pointer chasing.[7] The tree is a pointer-based octree, where each node stores a pointer to its first child (if it has one) and to the next encountered node in a depth-first scan (e.g., its next sibling).

The tree topology is constructed via iterative walking. If the leaf threshold is 1, the tree is walked until an empty child slot is found. If, instead, a leaf already containing a particle is found, then that node is split and, if necessary, its child containing the two particles is split, until each particle sits in a unique node. This can be generalized if the desired leaf threshold is greater than 1.

Once the topology has been fixed, an upward pass computes the multipole moments of the internal nodes.[8] This can be done in a number of ways, but one illustrative example is a postorder recursive traversal. Starting from the root, the routine descends through each node's children until it reaches the leaves, whose moments are computed directly from the particles they contain. As the recursion returns, each internal node combines the already-computed moments of its children to form its own multipole expansion. Applying this routine to the root therefore computes the moments of every node in the tree, proceeding implicitly from leaves to root.

Walking is then done simply by iteratively applying the MAC to each node, starting with the root node. The pointer to a node's first child is dereferenced when a node is opened, and the pointer to the next node in the depth-first scan is dereferenced when a node is closed.

The pointer-chasing technique is ill-suited to custom hardware solutions because of its irregular memory access pattern in both construction and walking. Of course, as mentioned before, the tree algorithm will always have some inherent irregularity in its memory access during walking. But in this case, the tree is repeatedly walked during the construction of its topology. Furthermore, the algorithm's branching logic presents additional problems.

2.3. *Breadth-First*

The incremental insertion of the pointer chasing technique is poorly suited to GPU-parallelization, which has motivated the technique proposed in `Bonsai` by J. Bédorf et al. (2012). The tree is stored breadth-first, where each level is contiguous in the array. Each node stores the index of its first child and its number of children, along with the usual multipole moments.

First, all particles are sorted along a SFC (Morton order in J. Bédorf et al. (2012)). In this order, particles belonging to the same node are always contiguous. Then, starting with the top level, each particle's SFC key is masked to identify the node that particle belongs to on that level. Particles with identical masked keys are grouped into nodes using parallel stream compaction. Any node containing no more than a threshold number

[5] A node's subdivisions are its children, nodes at the same level are siblings, and the node they are contained within one level higher is their parent.

[6] Subsequently, a relative opening criterion was proposed by V. Springel et al. (2001) comparing the truncation error computed from the next-highest multipole term to an estimate of the force's magnitude (e.g., from the previous timestep). For a given force accuracy requirement, this opening criterion requires less computational effort. See also J. K. Salmon & M. S. Warren (1994) for a discussion of worst-case error estimates from various opening criteria.

[7] See also https://home.ifa.hawaii.edu/users/barnes/treecode/treeguide.html

[8] One can also avoid the upward pass by iteratively updating each node's multipole moments during the particle insertion phase.

of particles (N_{leaf}, e.g., 16) is marked as a leaf, and its particles are ignored at subsequent levels. This process is repeated level-by-level until every particle is assigned to a leaf or a maximum depth is reached.

Then, a linking stage is performed. Every node is assigned a thread that finds the node's parent with a binary search in the key-sorted node array, incrementing the parent's child count; the same thread also finds the node's first child with a second binary search.[9] Because many threads concurrently update the same parent, the child-count updates require atomic read-modify-write operations. After this linking stage, a bottom-up pass computes multipole moments level-by-level, with one thread per node combining the moments of its children.

The tree is then walked breadth-first for groups of nearby particles rather than for individual ones, following Barnes' vectorization of the tree walk (J. E. Barnes 1990). Starting from a list of nodes at one of the top levels, each thread evaluates the MAC between one node and the whole group: accepted nodes are appended to a node-interaction list, rejected (opened) nodes push their children onto the next level's list, and leaves append their particles to a particle-interaction list. Whenever an interaction list becomes large enough, it is evaluated concurrently by all threads in the group. The walk proceeds level by level until all interactions have been accounted for.

3. STREAMING TREE ALGORITHM

Neither pointer-chasing nor breadth-first tree algorithms are well-suited to FPGAs. Pointer-chasing trees require dynamic memory allocation and random access during both construction and traversal. The breadth-first algorithm of J. Bédorf et al. (2012) requires per-level stream compaction passes, linking via binary searches with atomic updates, and group-based interaction-list management, all of which map poorly onto the fixed pipelines and limited area of an FPGA. Here we describe a new tree construction and traversal algorithm designed around the streaming, pipelined execution model natural to FPGAs.

3.1. *Tree Structure*

Our tree is stored as a depth-first linear array of node-leafs. Because we mix nodes and leaves in the array, we use a common struct that can represent either a node or leaf. The first element is one of the top-level nodes, which we fix at level 1 (i.e., the first subdivision of the domain into octants). The next element is that node's first child, then that child's first child, and so on down to a user-specified maximum depth D. All nodes at depth D are leaves, and every leaf contains exactly one particle. If two particles share the same Peano–Hilbert (PH) key down to depth D, they occupy separate leaves with identical PH keys. In this way, every particle's position is stored directly in the tree.

Duplicating particle data in the leaves introduces a modest storage overhead but improves data locality during traversal, since particle and node data are interleaved in a single contiguous array.

3.2. *Construction*

Input: sorted particles P, max depth D
Output: tree array T (depth-first order)
Initialize node stack $S[1..D]$ with $P[0]$;
for $i \leftarrow 1$ **to** $|P|$ **do**
  $\ell^* \leftarrow$ shallowest level where $P[i]$ diverges from S;
  if *no divergence* **then**
    $\ell^* \leftarrow D$; `// force new leaf`
  end
  Emit $S[\ell^*..D]$ to T (deepest first);
  Reset $S[\ell^*..D]$ with $P[i]$;
  Accumulate $P[i]$ into $S[1..\ell^*-1]$;
end
Flush $S[1..D]$ to T;
Reverse T;

Algorithm 1: Tree construction

FPGAs achieve high throughput through pipelining: conceptually, data streams through the device rather than being acted upon in place. Our construction algorithm is designed with this model in mind.

Sorting.—Particles are first sorted by their PH key, computed to the maximum depth D. A key property of PH ordering is that a particle's key encodes its membership in every node from the root down to depth D. For example, in 2D a key 00 01 10 00 indicates membership in the 00 node at level 1, the 00 01 node at level 2, and so on. Consequently, all particles belonging to any given node are contiguous in the sorted order.

Streaming accumulation.—A stack of D nodes is maintained in local memory. Each node tracks accumulated center-of-mass and multipole data. The stack is initialized with the properties of the first particle.

Particles are then processed sequentially. For each new particle, its PH key is compared against the keys of the nodes on the stack to determine the *divergence level*:

[9] By virtue of the particle list being initially sorted along a SFC, all of a node's children are contiguous. Therefore, to find all of a node's children requires only knowing the total number of children and the index of the first child.

the shallowest level at which the new particle's key differs from the current stack. Nodes shallower than the divergence level (i.e., those the new particle still belongs to) continue accumulating. All nodes at or deeper than the divergence level are finalized: they are emitted and appended to the output array, then reset and seeded with the new particle's properties. After the final particle has been processed, the remaining stack is flushed.

Sibling connections (i.e., the index of each node's next sibling) are recorded during emission, so no separate linking pass is required. Note that for leaf nodes, the next sibling is always just the next node in the list.

Output.—The result is a linear array of nodes in reverse depth-first order: the last node emitted is a top-level node. The array is reversed to yield the final tree. Because the tree is constructed in streaming fashion, it is also possible to begin traversal before construction is complete—for example, once one of the eight top-level octants has been fully emitted—though we leave this optimization to future work.

3.3. *Walking*

Input: particles P, tree nodes T
Output: accelerations $\mathbf{a}$
foreach *particle* $p \in P$ **do**
  $i \leftarrow 0$;
  $\mathbf{a}_p \leftarrow 0$;
  while $i \neq$ `SENTINEL` **do**
    $n \leftarrow T[i]$;
    if accept(p, n) ***or*** n.is_leaf **then**
      $\mathbf{a}_p \leftarrow \mathbf{a}_p + \text{force}(p, n)$;
      $i \leftarrow n$.next_sibling; `// node closed`
    else
      $i \leftarrow i + 1$; `// node opened`
    end
  end
end

Algorithm 2: Tree walk

Tree walking is a relatively straightforward operation, outlined in Algorithm 2. For each particle, we start with the first top-level node (the last node to be emitted during the tree construction process), the desired multipole acceptance criterion is evaluated for the particle-node pair. If the criterion is accepted or the node is a leaf, the particle accumulates the node's force contribution and the loop skips to the next sibling, whose index is stored with the node's data. If the criterion is rejected, the loop continues to the next index, which is guaranteed to be that node's first child. The process is iterated until all nodes have been considered.

4. DESIGN

We now summarize the details of how we have implemented our new BH tree algorithm on our hardware. Our design consists of two kernels—`treecon` for tree construction and `treewalk` for tree walking—which we describe in turn, followed by the data structures and number formats they share and their placement on the board.

4.1. *Hardware Setup*

Our design targets an AMD Alveo U200 (hosting the XCU200-L2FSGD2104E device) and runs at a post-route clock frequency of 100 MHz. This board has 64 GB of DDR4 RAM with a transfer rate of up to 2400 MT/s across four banks. Our source code is written in pure C++ HLS, which we compile into a bitstream using AMD's Vitis HLS toolchain version 2023.2.2. For continuous integration, we built a Docker image with the Vitis HLS toolchain installed. We cannot make this image public due to licensing requirements, but the steps we took to create it are documented in the `DOCKER_IMAGE.md` file. All tests were performed in a container based on this image.

Our host system, and the Docker image, runs Ubuntu 22.04.4 LTS. Our host has an Intel Xeon Silver 4216 CPU @ 2.10GHz with 16 cores (32 threads), and 128 GB of DDR4 ECC RAM running at 2400 MT/s. Our Alveo U200 is connected to the host over PCIe Gen3 at 8 GT/s across 16 lanes.

4.2. *Tree Construction Kernel*

On the FPGA, tree construction (Algorithm 1) is implemented as kernel `treecon`, a two-stage dataflow with subkernels `particle_processor` and `node_writer` connected by a set of parallel streams (one per node field). These subkernels perform the following tasks:

- `particle_processor`: Particles are read sequentially from DRAM. The stack of D partially accumulated nodes is held in registers, so all levels are operated on in parallel: each particle's PH key is compared against the stack keys at every level simultaneously to determine the divergence level, the mass-weighted position and mass accumulators of the surviving levels are updated, and the finalized nodes are emitted to the output streams. The sibling index that each node stores is updated for the stack during emission.

- `node_writer`: Completed nodes are drained from the streams. The center of mass is computed by dividing the accumulated mass-weighted position by

the node mass, the leaf flag is set if the node contains no more than N_{leaf} particles, and the finished 64-byte node is packed into a single 512-bit word and written to DRAM in one AXI beat. When the final top-level node has been written, the total node count is reported back to the host.

Splitting construction into two subkernels decouples the accumulation logic from the DRAM writes, with bursts of node emissions absorbed by the connecting streams. The emission loop in `particle_processor` is serialized rather than unrolled: a particle that finalizes k nodes spends k cycles emitting them. This saves the resources an unrolled emitter would require, at the cost of a per-particle initiation interval of D (the scheduler must assume the worst case in which the entire stack is flushed). As we will see in Section 5, tree construction takes far less wall time than tree walking, justifying this tradeoff.

4.3. *Tree Walking Kernel*

The tree walk loop of Algorithm 2 is implemented as kernel `treewalk` using a cyclic dataflow paradigm with five subkernels: `read_particles`, `work_distributor`, `force_kernel`, `read_kernel`, and `output_kernel`. These subkernels perform the following tasks:

- `read_particles`: Particles are read from DRAM and placed into a stream connected to `work_distributor`.
- `work_distributor`: This kernel places particles into the cyclic region by streaming them to `force_kernel`. Particles are read from a stream from `read_particles` (new particles) and from a stream from `read_kernel` (particles existing in the cyclic region). If a particle is available from both streams, priority is given to particles that are continuing in the region.
- `force_kernel`: Particles are read from a stream from `work_distributor`. The opening criterion is evaluated and, if necessary, the partial force contribution is accumulated. The particle, along with the next node that needs to be read, is passed to `read_kernel`. If the particle has reached the end of the tree, it is instead passed to `output_kernel`.
- `read_kernel`: Particles are accepted from `force_kernel` along with their next requested node. A DRAM request is made for this node and, when fulfilled, the particle is sent back to `work_distributor`.
- `output_kernel`: Particles from `force_kernel` with their completed accelerations are written to a DRAM buffer for the host to retrieve.

Each subkernel is pipelined with an II of 1, and so one interaction can be processed per clock cycle if the dataflow region can be continually fed. The total pipeline has a latency of 300 clock cycles TODO: finalize number, and so we need to have $\gtrsim 300$ particles in flight at a time in order to keep the pipeline full.

This walking algorithm is better suited to FPGAs than prior methods because it limits random memory accesses to just node closures – any time a node is opened or a leaf is encountered, the next node is always the next entry in the tree.

4.4. *Data Structures*

Both kernels exchange particles and nodes with DRAM as 64-byte records, sized to match one 512-bit AXI beat so that each access completes in a single memory transaction. The layouts are given in Tables 1 and 2, with the underlying number formats described in Section 4.5.

For particles (`particle_t`; Table 1), the position, mass, and softening index are inputs and the acceleration is the output. The PH key is computed on the host during sorting and consumed by `treecon`. The remaining fields carry per-particle state through `treewalk`'s feedback loop: `next_tree_idx` records the next node to be visited, while the three interaction counters tally node openings, leaf interactions, and node interactions, which we use to instrument the walk in Section 5.

A single structure (`nodeleaf_t`; Table 2) represents both nodes and leaves, which are mixed in the depth-first tree array. Each stores its center of mass and mass fraction, its level (which sets the node size in the opening criterion), and the sibling link used to skip over its subtree when it is closed. The `start_idx` and `num_particles` fields delimit the contiguous range of particles the node contains. The `hmax_idx` field stores the largest softening index of any particle in the node. Finally, the `is_last` flag marks the final node written during construction, which is also the entry point of the walk.

4.5. *Number Formats*

We now summarize the salient number formats of our design, with a full list given in Table 3. Because coordinates have values in $[0, 1)$, we store them in unsigned fixed point with N fractional bits and no integer bits, denoted `pos_t`. For this work we always assume $N = 32$, intermediate between the 23 and 52 mantissa bits of 32- and 64-bit floating point, respectively. This is equivalent to integer coordinates as in V. Springel et al.

Table 1. Particle data structure (`particle_t`).

Field	Type	Width (bits)	Description
`pos[3]`	`pos_t`	96	position
`acc[3]`	`acc_t`	96	acceleration
`mass`	`mass_t`	32	mass fraction
`idx`	`count_t`	32	particle index
`key`	`phkey_t`	48	Peano-Hilbert key
`next_tree_idx`	`count_t`	32	next tree index
`hidx`	`hidx_t`	6	softening table index
`Nint_opened`	`count_t`	32	node-opening count
`Nint_leaf`	`count_t`	32	leaf interaction count
`Nint_node`	`count_t`	32	node interaction count
padding		74	
Total		512 (64 bytes)	

Table 2. Node/leaf data structure (`nodeleaf_t`).

Field	Type	Width (bits)	Description
`pos[3]`	`pos_t`	96	center of mass / first particle position
`mass`	`mass_t`	32	mass fraction
`level`	`level_t`	8	tree depth
`start_idx`	`count_t`	32	index of first child/particle
`num_particles`	`count_t`	32	particle count
`next_sibling`	`count_t`	32	next sibling index
`is_leaf`	`flag_t`	1	leaf flag
`is_last`	`flag_t`	1	last sibling flag
`valid`	`flag_t`	1	validity flag
`hmax_idx`	`hidx_t`	6	max softening table index
padding		271	
Total		512 (64 bytes)	

(2021).[10] Storing coordinates in fixed point allows intermediate calculations to also use fixed precision with finite bounds, avoiding costly floating point operations. Particle and node masses are stored as unsigned fixed point fractions of the total box mass (`mass_t`), with convergent rounding and saturation overflow modes enabled (since the total mass may slightly exceed unity due to roundoff error). In this work we use the same number of fractional bits N for both `pos_t` and `mass_t`.

Quantities derived from the softening length, however, can require quite wide data types if implemented naively. For example, if the softening length h were represented with the `pos_t` type, then $1/h^3$ would need a 128-bit representation, and additions and multiplications with such wide types are expensive. Floating point is the natural solution, but is itself resource intensive on hardware, largely because the IEEE-754 standard requires several features like normalization (keeping the mantissa in $[1, 2)$), rounding to even, and special case handling (NaN and infinity).

In our case, we require some of the dynamic range of floating point but not necessarily all of the IEEE-754 features and guarantees. Instead, we use a custom floating point architecture with similar precision to `float32`. All intermediate calculations are unnormalized and no special case checking is done, which is valid because we know the range of all quantities involved. We also truncate instead of round, carrying one additional bit of mantissa precision to achieve the same 0.5 ULP precision as `float32`. This leads to a much lower resource usage than standard floating point.

4.6. *Force Evaluation*

Rather than passing softening lengths as arbitrary numbers, we construct 64-entry softening tables—one

[10] SFC ordering keeps consecutive particles close in space, so the memory footprint could be reduced by storing only coordinate differences between successive particles, or via the more sophisticated scheme of H.-R. Yu et al. (2018).

Table 3. Data types used in the force calculation. Note that the widths of the `phkey_t` and `level_t` types depend on the chosen maximum depth D.

Type	Definition	Description
`pos_t`	`ap_ufixed<32,0>`	coordinates
`mass_t`	`ap_ufixed<32,0,AP_RND_CONV,AP_SAT>`	mass (fraction of total)
`acc_t`	`float`	acceleration (output)
`hidx_t`	`ap_uint<6>`	softening table index
`phkey_t`	`ap_uint<48>`	Peano-Hilbert key
`level_t`	`uint8_t`	tree depth (max 64)
`count_t`	`uint32_t`	indices and counters
`flag_t`	`ap_uint<1>`	boolean flag

for h^{-2} and one for h^{-3}—in which successive entries are spaced by a factor of 1.2. The softening length of each particle and node is then stored as a 6-bit unsigned integer key (`hidx_t`), and min/max comparisons can be done directly on the integer, which is significantly cheaper than fixed- or floating-point comparators. The table entries themselves are stored in our custom floating point format with a 6-bit unsigned exponent (unsigned since we always assume $h < 1$ and store inverses).[11] As the force calculation proceeds, this exponent is progressively widened to avoid overflow.

Instead of directly computing the partial forces, which would require an inverse square root and many multiplications within the softened regime, we perform a direct interpolation. This means that, for a given particle-particle or particle-node interaction, we compute the partial force as

$$\mathbf{a}_{ij} = m_j \, h_{ij}^{-3} \, g(u_{ij}) \, (\mathbf{x}_j - \mathbf{x}_i), \tag{2}$$

where m_j is the mass of particle/node j, h_{ij} is the pairwise softening, u_{ij} is the distance between i and j normalized by the softening length, and $g(u)$ is the dimensionless, softened force law. In GADGET notation (V. Springel 2005), our g is simply $-h^3 g_1$. Because computing u requires a square root, we tabulate g as a function of u^2 instead. We compute r^2 in fixed point, convert to floating point, and then multiplying by the tabulated value of h^{-2} to obtain u^2. This last operation is cheap, requiring one mantissa multiplication and one exponent addition (both fixed-point operations).

Our interpolation table for g stores values only for $u^2 \in [0, 4)$. The softened force law applies for $u^2 < 1$, and when $u^2 > 4$, we simply rescale by powers of 4 to be within $u^2 \in [1, 4)$, using the fact that $g = 1/u^3$ for $u > 1$ to scale back at the end (both scalings are exponent manipulations). We then multiply by the mass of the particle/node and by the tabulated value of h^{-3}. The partial force is converted to an IEEE-754 `float32` and then accumulated. We normalize only when computing position differences, when computing u^2, before multiplying by the node/particle mass, and at the end when converting to `float32`. The full sequence of operations, with the conversions between representations, is summarized in Table 4.

[11] This corresponds to a maximum allowed value for h^{-3} of 2^{64}, or a minimum value of h of $\sim 3.8 \times 10^{-7}$ (in units of the box size). For reference, for a 50 Mpc box, this corresponds to a softening of ~ 19 pc. Smaller softenings can be achieved by widening this exponent.

4.7. *Configuration*

The U200 has three super logic regions (SLRs) and four DDR banks, with each bank physically close to one SLR. Crossing an SLR boundary requires traversing a super long line (SLL), which adds routing delay and is a common source of timing closure difficulties at high clock frequencies. We therefore place each kernel and its associated memory interfaces on a single SLR whenever possible.

We place the `treecon` kernel on SLR0 and the `treewalk` kernel on SLR1. Both kernels are clocked at 100 MHz. Three of the four DDR banks are used. The input particle buffer resides in DDR0 and is read by both kernels: first by `treecon` during streaming tree construction, and again by `treewalk` as the source of the particles whose accelerations are to be computed. The tree itself is written by `treecon` to DDR1 and is read back by `treewalk` during traversal. The output accelerations are written by `treewalk` to DDR2. DDR3 is unused.

This assignment is motivated by the asymmetry in memory traffic between the two kernels. The `treewalk` kernel is latency-bound on its tree AXI port and thus benefits most from a low-latency path to the tree, so we co-locate it with its heavily-used DDR bank on SLR1. This means that the `treecon` kernel crosses a SLL to write the tree. However, because the tree write is sequential and `treecon` is a subdominant component of the runtime anyways, this SLL crossing is tolerable.

Table 4. Force calculation for particle p and node n. Intermediate quantities are held in our custom mantissa/exponent floating point format (denoted m/e), in which mantissas are unnormalized except where noted and no rounding or special case handling is performed until the final cast to `float32`.

Computation		Conversion	Note
Δx	$\leftarrow x_n - x_p$	`pos_t` $\rightarrow$ fixed	also for Δy and Δz
r^2	$\leftarrow \Delta x^2 + \Delta y^2 + \Delta z^2$	fixed $\rightarrow$ m/e	normalized
k	$\leftarrow \max(k_n, k_p)$	—	pairwise softening; 6-bit integer compare
h^{-2}, h^{-3}	$\leftarrow$ `soft_table`$[k]$	—	stored as m/e pairs
u^2	$\leftarrow r^2 \cdot h^{-2}$	—	exponent gives exact $u^2 < 4$ test
g	$\leftarrow$ `g_lut`(u^2)	—	1024-entry LUT + lin. interp.; u^2 is rescaled when $u^2 \geq 4$
f	$\leftarrow m_n \cdot h^{-3} \cdot g$	`mass_t` $\rightarrow$ m/e	mass normalized first
$\mathbf{a}_p$	$\mathrel{+}= f \cdot \Delta\mathbf{x}$	m/e $\rightarrow$ `float32`	$\Delta\mathbf{x}$ normalized; sign carried separately

There is also an SLL crossing when `treewalk` reads from the particle list, but again this read is sequential and each particle requires multiple trips through the `treewalk` data loop, so the higher latency and lower bandwidth incurred by the SLL crossing is tolerable.

5. PERFORMANCE

We now turn to summarizing the performance of our design. We test our design on two setups with varying particle numbers and distributions: a uniform density distribution and a Hernquist sphere, the latter which intends to test the design on a clustered distribution. In each case, we vary the particle number by factors of 4 from 2^{12} to 2^{20} (from $\sim$ 4k to $\sim$ 1M). For the Hernquist sphere, we also test varying its scale length a, varying from 0.001 to ∞ (i.e., a uniform sphere of radius 0.5). In all cases we average over three random seeds for each distribution.

We first demonstrate the accuracy of our design with respect to a direct sum calculation before turning to the design's throughput.

5.1. *Accuracy*

We show in Figure 1 the relative force error as measured between the FPGA computed value and the value from direct summation. The 90th percentile and mean values computed from a random subsample of 4096 particles are shown in solid and dashed lines, respectively.

In general, the force accuracy is better in the Hernquist sphere case than for the uniform distribution. The 90th percentile and mean errors in the Hernquist sphere are no larger than $\sim 0.7\%$ and $\sim 0.4\%$, respectively. For the uniform distribution these values are slightly larger at $\sim 1\%$ and $\sim 0.5\%$. In the uniform distribution case the error improves with larger particle numbers while it remains roughly constant in the Hernquist case.

In Figure 2, we show how the relative force error scales with opening angle θ. Because the dipole moment vanishes, the leading order error term is expected to be $\propto \theta^3$

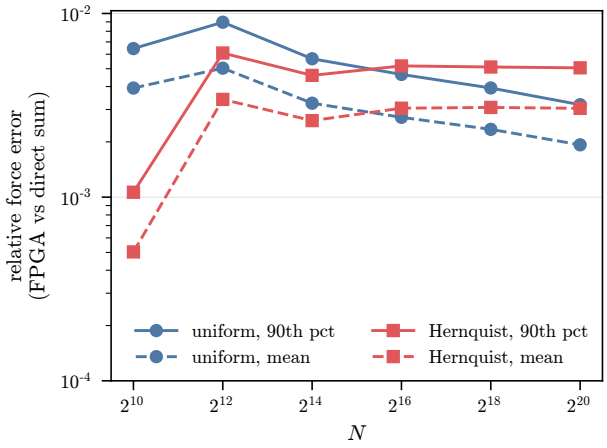

Figure 1. Error scaling with particle number for our uniform (blue circles) and Hernquist ($a = 0.5$; red squares) distributions. We show the 90th percentile and mean errors in solid and dashed lines, respectively. These relative errors statistics, computed against a direct summation estimate, all lie below 1%.

(e.g. V. Springel et al. 2021), which we plot as a dashed line. The 90th percentile relative error follows this scaling reasonably well, with some deviation at the low-θ end.

We have also verified that our FPGA and software implementations give nearly identical acceleration values. For the same 4096 particle subsample, the maximum deviation between the two implementations is $\sim 0.28\%$ or $\sim 0.03\%$ in the uniform/Hernquist sphere cases, respectively. In both cases, 99% of particles have a relative difference between the two of less than 0.0032%.

The concordance between our FPGA algorithm, direct summation, and our software implementation of the algorithm, as well as the expected θ^3 scaling, gives us confidence that our design is computing forces accurately.

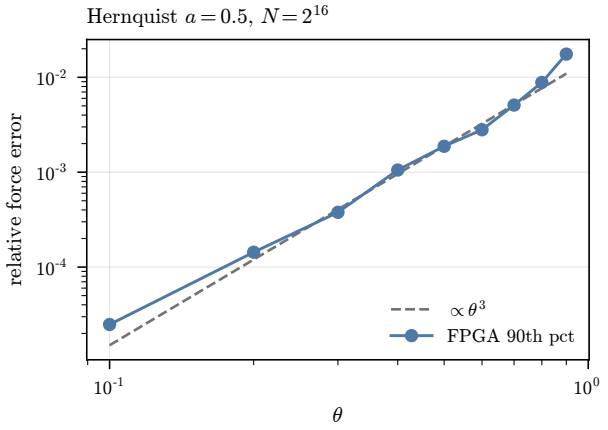

Figure 2. The 90th percentile relative error as a function of opening angle θ, for the distribution with $a = 0.5$ and 2^{16} particles ($\sim 65k$). The error value follows the expected θ^3 scaling (gray dashed line anchored to $\theta = 0.5$) except close to $\theta = 1$ and $\theta = 0.1$. At the lower end, the deviation results from the FPGA's use of a fixed-point representation (with 32 fractional bits) for the particle positions, while the direct summation reference calculation uses a double representation, which has no fewer than 53 fractional bits between 0 and 1.

5.2. *Total Wall Time*

Having demonstrated the accuracy of our FPGA implementation, we now turn to its performance. In Figure 3 we show the total wall time for the `treewalk` and `treecon` kernels in solid and dashed lines, respectively. As before, the uniform and Hernquist $a = 0.5$ distributions are shown in blue circles and red squares, respectively. These are all shown for varying particle number N. Note that the `treecon` lines are nearly overlapping for the two distributions, indicating that our `treecon` algorithm's performance is independent of the particle distribution. The `treewalk` kernel, on the other hand, does depend on the particle distribution, with the more clustered Hernquist distribution (which requires more leaf interactions) taking about four times as long as the uniform distribution. The `treewalk` wall times closely follow the expected $N \log N$ scaling, shown as a black dotted line.

Table 5 shows these scalings explicitly. For each particle number, we list the `treecon` wall time divided by N and the `treewalk` wall time divided by $N \log_2 N$, for both distributions. Each column is relatively constant at larger N, demonstrating the $\mathcal{O}(N)$ scaling of tree construction and the $\mathcal{O}(N \log N)$ scaling of tree walking.

Table 5. Kernel wall times normalized by their expected scalings: `treecon` wall time divided by N, and `treewalk` wall time divided by $N \log_2 N$, for the uniform and Hernquist ($a = 0.5$) distributions. Host overhead is presumably responsible for the elevated `treecon` entries at the smallest particle numbers.

	`treecon`/N [ns]		`treewalk`/$(N \log_2 N)$ [ns]	
N	uniform	Hernquist	uniform	Hernquist
2^{12}	1204	1016	492	1888
2^{14}	249	348	635	2825
2^{16}	198	204	558	2709
2^{18}	194	196	598	3239
2^{20}	193	193	526	2756

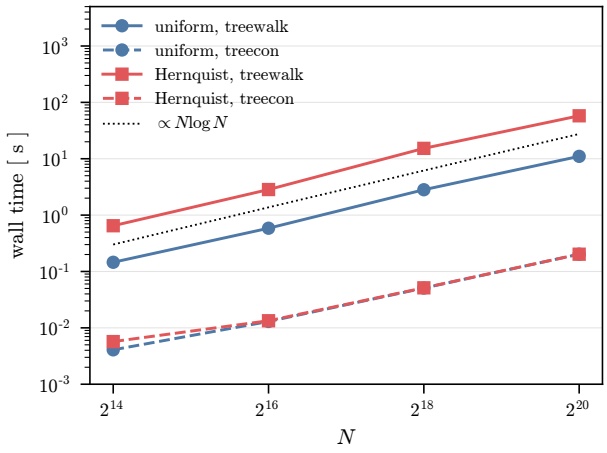

Figure 3. The scaling of our algorithm with problem size. The wall time for the uniform and Hernquist $a = 0.5$ distributions is given in blue circles and red squares, respectively. The `treewalk` and `treecon` kernels are shown in solid and dashed lines, respectively. Note that the `treecon` lines almost completely overlap between the two distributions. TODO: Make treewalk/treecon in figure tt. Add $N = 2^{12}$ and 2^{10}?

5.3. *Tree Construction Throughput*

We show the throughput for the `treecon` kernel in Figure 4 for the uniform distribution scaling with particle number. Because the serialized node emission loop gives the kernel an II of $D = 16$ (Section 4.2), the theoretical maximum throughput at 100 MHz is 6.25 particles μs^{-1}. The design achieves a throughput of about 5 particles μs^{-1} at all particle numbers. As shown in Figure 3, the resulting wall time of `treecon` is well below that of `treewalk`.

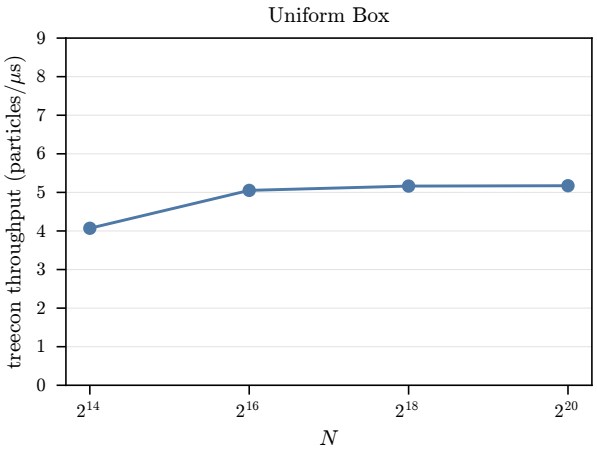

Figure 4. The scaling of `treecon` with particle number for the uniform box. At all particle numbers, the throughput saturates at around 5 particles μs^{-1}, close to the theoretical maximum of 6.25 particles μs^{-1}. TODO: Add 6.25 line.

5.4. *Tree Walking Throughput*

We next show the throughput for the `treewalk` kernel in Figure 5. On the left panel we show the uniform distribution and on the right panel we show the Hernquist $a = 0.5$ sphere. In this case, our II is 1, and so the maximum throughput is 100 μs^{-1}, shown as a horizontal dashed line. We show the average number of node interactions, leaf interactions, and node openings per μs as blue squares, orange triangles, and green diamonds, respectively. The total throughput, combining these three numbers, is shown as black circles.

First, the total maximum throughput is high, achieving $> 90\%$ of the maximum for nearly all particle numbers. In both distributions, the number of node interactions increases as the number of leaf interactions decreases.

In Figure 6, we vary the scale length of the Hernquist distribution, thus varying the amount of clustering (smaller a is more clustered). The upper panel shows the throughput of the design, where the various lines are the same as in Figure 5. The maximum throughput remains high even as $a \to 0$ and the particle distribution becomes more concentrated. In the bottom panel, we also show that the more concentrated distribution has a more accurate force estimate, which could be explained by the higher fraction of leaf interactions compared to node interactions shown in the upper panel.

5.5. *Resource Usage*

We show the final resource usage in Table 6 for our design. Both kernels are small compared to the available resources. The `treewalk` kernel's largest fractional usage is its BRAM, consuming 29 of the 1793 BRAM tiles available to user logic, with LUTs close behind and DSPs well below either. This implies that our `treewalk` design could be replicated on the board approximately 60 times, and approximately 80 times on the larger U250 board, which hosts the XCU250-L2FIGD2104E device. This is discussed further in Section 6.1.

6. DISCUSSION

6.1. *Comparison to CPUs and GPUs*

We showed in Section 5.4 that our FPGA implementation is able to achieve $\gtrsim 90\%$ of the theoretical maximum throughput of 100 it/μs (where interactions includes node openings). We tested our software implementation of the same algorithm and found that our CPU was able to reach $\sim$ 125 it/μs.[12] The number of `treewalk` kernels that can be replicated thus determines the competitive balance between a CPU and FPGA solution.

Our `treewalk` design is resource-limited by its BRAM usage.[13] The `treecon` kernel and shell leave 1 780 BRAM tiles for additional kernels, and so this implies ~ 60 additional kernels could fit onto the board. Some overhead needs to be left for the compiler, and so we assume ~ 50 additional kernels can fit onto the U200. The larger U250 board has about 1.25× as many BRAM tiles and 1.5× as many LUTs, and so we assume ~ 80 `treewalk` kernels could fit on it.[14]

On the CPU side, modern clusters have nodes with core counts exceeding 128. Assuming the optimistic case for CPUs, a single node with 128 cores operating at $\sim$ 125 it/μs and achieving perfect parallelism would be capable of a throughput of 1.6×10^4 it/μs. In order to match this performance, we would need 160 `treewalk` kernels at our design's 100 MHz clock speed. If we pushed our design to 300 MHz (the maximum clock speed of the U200 board), we would only need 53 `treewalk` kernels, which is comparable to what a single U200 can deliver.

[12] We tested using `gcc v16.1.0` on our local desktop in addition to `gcc v15.2.0` on the Torch cluster at New York University. Our desktop has an Intel Xeon Silver 4216 CPU @ 2.10GHz (hyperthreading enabled) and the node we tested on with Torch has an Intel Xeon Platinum 8592+ CPU (hyperthreading disabled). The quoted number is for the Torch CPU, which was faster.

[13] Most of the `treewalk` kernel's BRAM usage comes from FIFOs between dataflow regions, and so can easily be absorbed into the available URAM blocks. In that case, LUTs become the limiting resource, but at a similar replication factor of ~ 70.

[14] See the Alveo UG1120 from 2023-10-11 https://docs.amd.com/r/en-US/ug1120-alveo-platforms/U250-Gen3x16-XDMA-4_1-Platform.

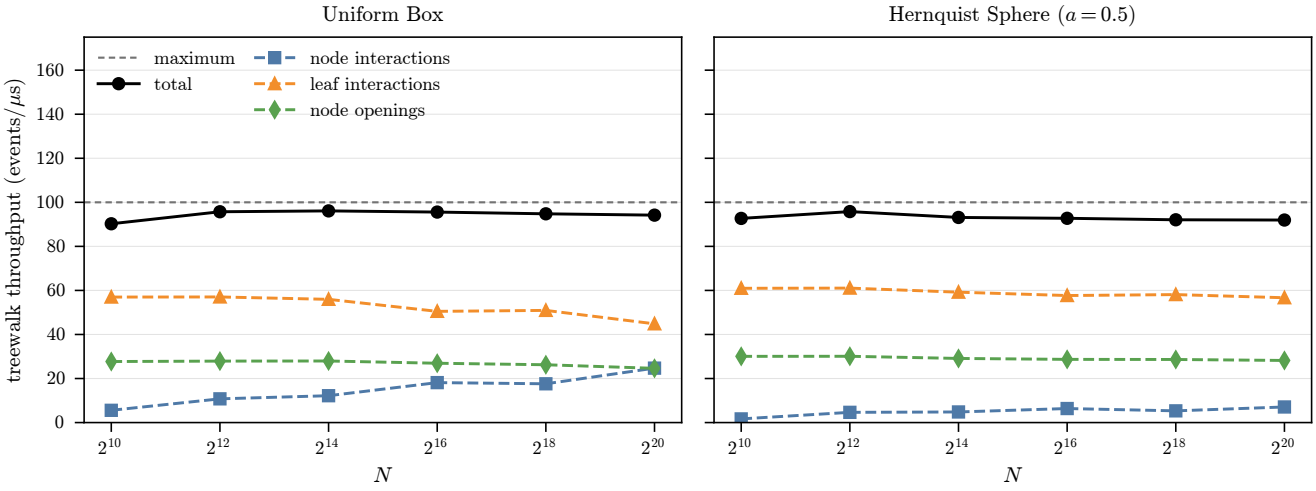

Figure 5. The total `treewalk` throughput as a function of particle number N for the uniform (left) and Hernquist $a = 0.5$ distributions. The solid black line shows the total throughput including interactions and node openings, which is always $> 90\%$ of the maximum of $100\,\mu s^{-1}$. In both cases, as the particle number increases the number of node interactions increases and the number of leaf interactions decreases, although the effect is more subtle in the Hernquist sphere case.

Table 6. Post-route resource utilization on the Alveo U200 for the combined dual-kernel design. TODO: Finalize these numbers.

	LUT	FF/REG	BRAM	DSP
`treecon`	44 796 (4.67%)	33 196 (1.62%)	13 (0.73%)	16 (0.23%)
`treewalk`	12 886 (1.34%)	15 058 (0.74%)	29 (1.62%)	42 (0.62%)
Kernel total	57 682 (6.02%)	48 254 (2.36%)	42 (2.34%)	58 (0.85%)
Full design	281 350 (23.80%)	369 848 (15.64%)	410 (18.98%)	71 (1.04%)
Available	1 182 240	2 364 480	2 160	6 840

TODO: Add GPU comparison, ask Marco.

6.2. *Future Improvements*

In this work, we have focused on a proof-of-concept implementation of our hardware design. We have shown that our design is accurate and meets our performance goals. We now discuss several areas for future improvement, beginning with raw performance and then turning to algorithmic extensions.

The most direct path to higher performance is headroom in the existing design. We targeted a clock speed of 100 MHz, while the maximum supported by the U200 is 300 MHz; increasing the clock speed typically requires longer compilation times, and so we did not pursue it here. Beyond clock speed, multiple `treewalk` kernels could be placed on the board—we estimated in Section 6.1 that ~ 50 would fit. The principal challenge is keeping that many kernels fed from the available DRAM banks: 50 kernels each requesting 512 bits per cycle at 300 MHz would require a bandwidth of $960\,\mathrm{GB\,s^{-1}}$, about 12.5× the DRAM bandwidth of the U200. However, the situation is not so dire because the memory requests are highly redundant. The upper levels of the tree are needed by almost every particle, and since particles are streamed to the kernels in SFC order, successive requests are often for the same nodeleaf (c.f. J. E. Barnes 1990). A cache system in which the upper levels and the most recently requested nodeleafs are stored in SRAM would therefore dramatically reduce the design's bandwidth requirements without introducing excessive complexity.

In a many-kernel design, tree construction would eventually become the bottleneck. Our results confirm that `treecon` is currently more than an order of magnitude faster than `treewalk`, validating the serialized node emission tradeoff made in Section 4.2, but this margin would be consumed as walking is parallelized. There are two paths forward. The first is to reduce `treecon`'s II toward 1 by separating node closure from node writing; we prototyped such a design and found it functional, at the cost of increased resource usage. The second is to parallelize construction itself across multiple `treecon` kernels. The particle stream could be split at PH-key boundaries at some upper tree level, with each kernel

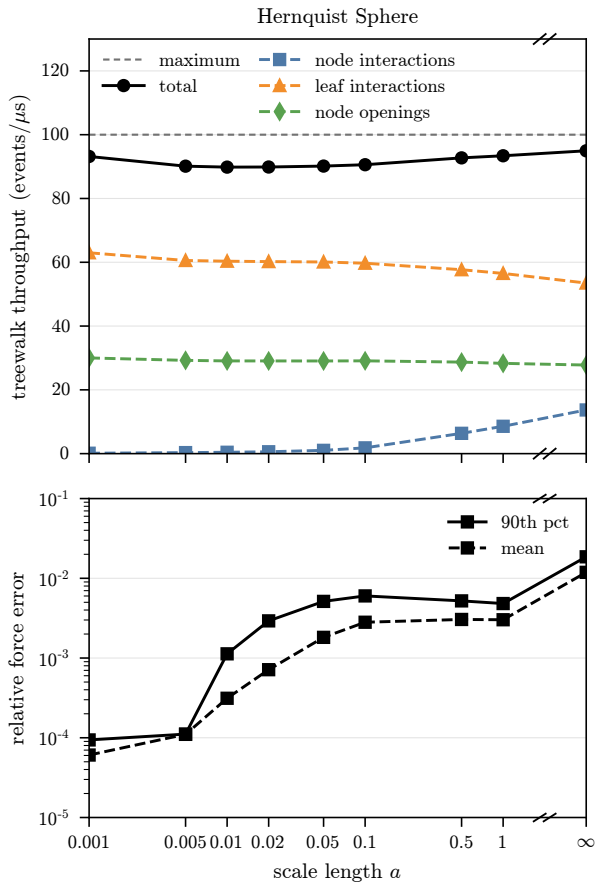

Figure 6. Error and `treewalk` throughput for the Hernquist sphere scaling with scale radius a. The upper panel shows the total throughput (black solid line) as well as the various components in the same style as Figure 5. As the distribution becomes more concentrated ($a \to 0$), the number of leaf interactions relative to the number of node interactions declines. This is accompanied by a more accurate force estimate (shown as the mean and 90th percentile relative errors in the solid and dashed lines in the lower panel). At all scale lengths, the total throughput remains close to the maximum throughput.

building the subtree beneath one upper-level node and the full tree stitched together afterwards. Alternatively, particles could be distributed cyclically across k kernels, each building a tree from its own subset, with the partial trees combined pairwise by a binary tree of $k-1$ merging kernels. The latter scheme guarantees load balance without any host-side domain decomposition.

Another avenue is to eliminate the remaining host-side work. In our design, sorting along the SFC is done on the host. In target cases where gravity is coupled to, e.g., hydrodynamics, this is probably not a huge issue since the particle data needs to be transferred onto and off of the board anyways. However, for a pure gravity simulation, it would be desirable to do this sorting on the board to eliminate this communication. Efficient sorting algorithms for FPGAs already exist (e.g., radix sorting), but given the specific drift pattern of astrophysical simulations, custom sorting algorithms may be beneficial.

On the algorithmic side, we only consider a simple geometric opening criterion in this work. As discussed in V. Springel (2005), a relative opening criterion in which the roundoff error is computed and compared to the acceleration from the previous timestep performs better in cosmological simulations with strongly clustered particle distributions. Our method could be extended to include this criterion with a relatively modest expansion in board area since, for the monopole approximation, it requires three multiplications – two to compute $|\mathbf{a}_{\rm old}|r^4$ and one to compute Ml^2/α – and one comparator. Further, since this approximation only affects the decision to open or close a node, the old acceleration can be stored and this computation can be done with narrower bit-widths.

We have also focused on non-periodic systems. In cosmological simulations one must additionally account for the periodicity of the box, which in a tree-only mode is typically handled by Ewald summation of the periodic images in the form of tabulated correction tables (P. P. Ewald 1921; L. Hernquist et al. 1991). The three 65^3 trilinear interpolation tables used by GADGET-2 would occupy 68 URAM blocks at 24 bits per entry, so 11 independent copies would fit on the U200. By time-multiplexing each copy across several `treewalk` kernels—at a cost of only a few clock cycles of latency—these 11 copies could serve $\sim$ 66 kernels. This would nevertheless entail a relatively large design change, and one would first need to weigh it against using the hybrid TreePM method, where the Ewald correction is not necessary (V. Springel 2005).

Finally, our streaming construction and walking paradigm could be relatively straightforwardly extended to the FMM. This would require more sophisticated control logic within the `treewalk` kernel as now chunks of the particle array would need to be streamed, but this may actually be a better match to our design as it increases the streaming fraction of the data. TODO: Add more details/understand better.

7. CONCLUSIONS

We have introduced the first, to our knowledge, implementation of the Barnes-Hut gravity tree on an FPGA in which both tree construction and walking are performed on the board.

Our implementation of the BH algorithm requires a strict ordering along a digit-recursive space filling curve, such as Morton Z-order or Peano-Hilbert. Then, tree construction occurs as a single streaming pass through the particles. The tree is then stored as a linear, depth-first array. For a given particle, walking is a one-way traverse through this array with skips during node closures (a MAC acceptance).

Given that FPGA development has not been commensurate with CPU and GPU advances, it is unlikely that an FPGA solution will compete with general purpose hardware in the near future. However, our novel design is a necessary advance for future development of gravity trees fully native on ASICs.

ACKNOWLEDGMENTS

Ack.

AUTHOR CONTRIBUTIONS

All authors contributed equally to the Terra Mater collaboration.

APPENDIX

A. APPENDIX

Appendix.

REFERENCES

Aarseth, S. J. 1999, PASP, 111, 1333, doi: 10.1086/316455

Aarseth, S. J. 2003, Gravitational N-Body Simulations

Barnes, J., & Hut, P. 1986, Nature, 324, 446, doi: 10.1038/324446a0

Barnes, J. E. 1990, Journal of Computational Physics, 87, 161, doi: 10.1016/0021-9991(90)90232-P

Bédorf, J., Gaburov, E., Fujii, M. S., et al. 2014, in Proceedings of the International Conference for High Performance Computing, 54–65, doi: 10.1109/SC.2014.10

Bédorf, J., Gaburov, E., & Portegies Zwart, S. 2012, Journal of Computational Physics, 231, 2825, doi: 10.1016/j.jcp.2011.12.024

Belleman, R. G., Bédorf, J., & Portegies Zwart, S. F. 2008, NewA, 13, 103, doi: 10.1016/j.newast.2007.07.004

Cheng, H., Greengard, L., & Rokhlin, V. 1999, Journal of Computational Physics, 155, 468, doi: 10.1006/jcph.1999.6355

Davé, R., Anglés-Alcázar, D., Narayanan, D., et al. 2019, MNRAS, 486, 2827, doi: 10.1093/mnras/stz937

Dehnen, W. 2000, ApJL, 536, L39, doi: 10.1086/312724

Dehnen, W. 2014, Computational Astrophysics and Cosmology, 1, 1, doi: 10.1186/s40668-014-0001-7

Ewald, P. P. 1921, Annalen der Physik, 369, 253, doi: 10.1002/andp.19213690304

Gaburov, E., Bédorf, J., & Portegies Zwart, S. 2010, Procedia Computer Science, 1, 1119, doi: 10.1016/j.procs.2010.04.124

Garrison, L. H., Eisenstein, D. J., Ferrer, D., Maksimova, N. A., & Pinto, P. A. 2021, MNRAS, 508, 575, doi: 10.1093/mnras/stab2482

Greengard, L., & Rokhlin, V. 1987, Journal of Computational Physics, 73, 325, doi: 10.1016/0021-9991(87)90140-9

Hamada, T., Fukushige, T., Kawai, A., & Makino, J. 2000, PASJ, 52, 943, doi: 10.1093/pasj/52.5.943

Hamada, T., Narumi, T., Yokota, R., et al. 2009, in 2009 International Conference on Sustainable Power Generation and Supply (IEEE), 62, doi: 10.1145/1654059.1654123

Hernquist, L., Bouchet, F. R., & Suto, Y. 1991, ApJS, 75, 231, doi: 10.1086/191530

Hockney, R. W., & Eastwood, J. W. 1981, Computer Simulation Using Particles

Holmberg, E. 1941, ApJ, 94, 385, doi: 10.1086/144344

Hopkins, P. F., Wetzel, A., Kereš, D., et al. 2018, MNRAS, 480, 800, doi: 10.1093/mnras/sty1690

Kawai, A., & Fukushige, T. 2006, in Proceedings of the 2006 ACM/IEEE Conference on Supercomputing, SC '06 (New York, NY, USA: Association for Computing Machinery), 48–es, doi: 10.1145/1188455.1188505

Klypin, A. A., & Shandarin, S. F. 1983, MNRAS, 204, 891, doi: 10.1093/mnras/204.3.891

Makino, J. 1991, PASJ, 43, 621, doi: 10.1093/pasj/43.4.621

Makino, J., Fukushige, T., Koga, M., & Namura, K. 2003, PASJ, 55, 1163, doi: 10.1093/pasj/55.6.1163

Nyland, L., Harris, M., & Prins, J. 2004, in GP2: ACM Workshop on General-Purpose Computing on Graphics Processors, Los Angeles, California

Okumura, S. K., Makino, J., Ebisuzaki, T., et al. 1993, PASJ, 45, 329, doi: 10.1093/pasj/45.3.329

Pillepich, A., Springel, V., Nelson, D., et al. 2018, MNRAS, 473, 4077, doi: 10.1093/mnras/stx2656

Portegies Zwart, S. F., Belleman, R. G., & Geldof, P. M. 2007, NewA, 12, 641, doi: 10.1016/j.newast.2007.05.004

Salmon, J. K., & Warren, M. S. 1994, Journal of Computational Physics, 111, 136, doi: 10.1006/jcph.1994.1050

Sano, K., Abiko, S., & Ueno, T. 2017, in Proceedings of the 8th International Symposium on Highly Efficient Accelerators and Reconfigurable Technologies, HEART '17 (New York, NY, USA: Association for Computing Machinery), doi: 10.1145/3120895.3120909

Schaller, M., Borrow, J., Draper, P. W., et al. 2024, MNRAS, 530, 2378, doi: 10.1093/mnras/stae922

Springel, V. 2005, MNRAS, 364, 1105, doi: 10.1111/j.1365-2966.2005.09655.x

Springel, V., Pakmor, R., Zier, O., & Reinecke, M. 2021, MNRAS, 506, 2871, doi: 10.1093/mnras/stab1855

Springel, V., Yoshida, N., & White, S. D. M. 2001, NewA, 6, 79, doi: 10.1016/S1384-1076(01)00042-2

Sugimoto, D., Chikada, Y., Makino, J., et al. 1990, Nature, 345, 33, doi: 10.1038/345033a0

Vogelsberger, M., Marinacci, F., Torrey, P., & Puchwein, E. 2020, Nature Reviews Physics, 2, 42, doi: 10.1038/s42254-019-0127-2

von Hoerner, S. 1960, ZA, 50, 184

Wang, L., Spurzem, R., Aarseth, S., et al. 2015, MNRAS, 450, 4070, doi: 10.1093/mnras/stv817

White, S. D. M., Frenk, C. S., & Davis, M. 1983, ApJL, 274, L1, doi: 10.1086/184139

Xu, G. 1995, ApJS, 98, 355, doi: 10.1086/192166

Yu, H.-R., Pen, U.-L., & Wang, X. 2018, ApJS, 237, 24, doi: 10.3847/1538-4365/aac830